

EXPERIMENT – 7

Performing Code Injection on Application Process Using Ptrace in Kali Linux

Aim

To demonstrate code injection on a running application process using the ptrace system call in Kali Linux for understanding low-level process manipulation and security vulnerabilities.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Programming Language:** C
- **Debugger / System Tool:** Ptrace
- **Compiler:** GCC
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

Code injection is a technique where malicious code is inserted into a running process to alter its execution. ptrace is a Linux system call that allows one process to observe and control another process, commonly used by debuggers. Understanding ptrace-based code injection helps analyze malware behavior, reverse engineering techniques, and system-level security vulnerabilities. Kali Linux provides a secure environment to study such low-level attacks ethically.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Create a Target Program

1. Open Terminal.
2. Create a simple C program:
3. nano target.c
4. Write a basic infinite loop program.

```
#include <stdio.h>

int main() {
    while(1) {
        printf("This is an infinite loop...\n");
    }
    return 0;
}
```

5. Compile the program:

6. gcc target.c -o target

7.

Step 3: Run the Target Application

1. Execute the program:

2. ./target

3. Note the **Process ID (PID)** using:

4. ps aux | grep target

Step 4: Create Injector Program

1. Save the file.

Step 5: Compile the Injector Program

1. Compile the injector:

2. gcc injector.c -o injector

Step 6: Attach to Target Process

1. Attach injector to target process using PID:

2. sudo ./injector <PID>

3. Injector gains control of the target process.

Step 7: Inject and Execute Code

1. Modify target process memory or registers.

2. Resume execution of the target process.

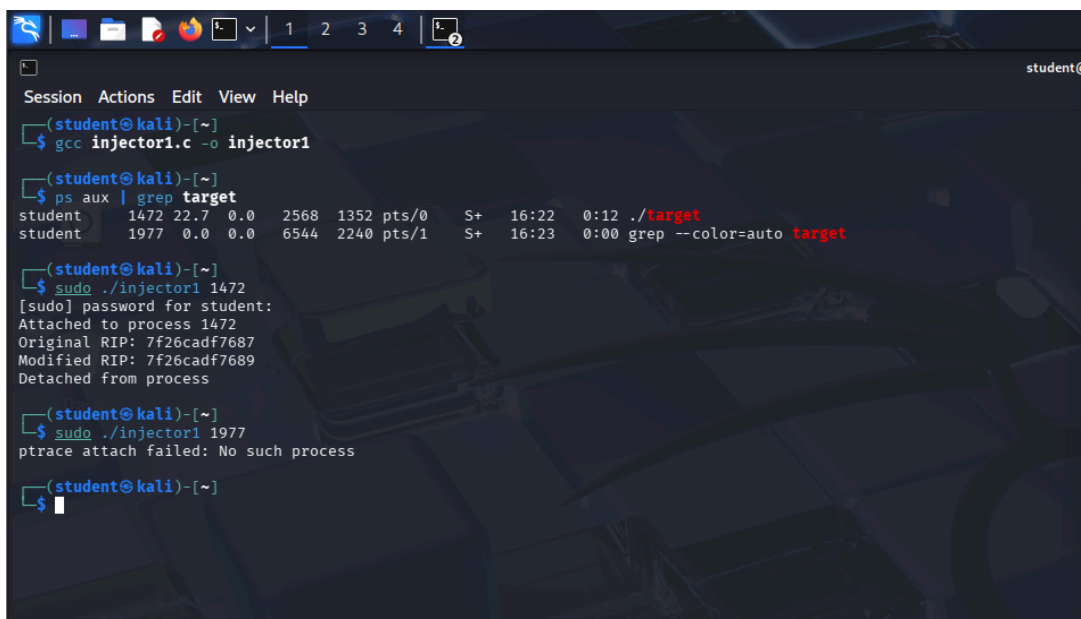
3. Observe injected behavior.

Output

- Injector successfully attaches to the running process.
- Target application behavior is modified during execution.
- Code injection is observed in real time.

Result

Code injection using the ptrace system call was successfully demonstrated in Kali Linux. The experiment illustrates how process memory and execution flow can be manipulated, emphasizing the importance of secure process isolation.



```
(student@kali)-[~]
$ gcc injector1.c -o injector1

(student@kali)-[~]
$ ps aux | grep target
student 1472 22.7 0.0 2568 1352 pts/0 S+ 16:22 0:12 ./target
student 1977 0.0 0.0 6544 2240 pts/1 S+ 16:23 0:00 grep --color=auto target

(student@kali)-[~]
$ sudo ./injector1 1472
[sudo] password for student:
Attached to process 1472
Original RIP: 7f26cadf7687
Modified RIP: 7f26cadf7689
Detached from process

(student@kali)-[~]
$ sudo ./injector1 1977
ptrace attach failed: No such process

(student@kali)-[~]
$
```